

Transformers & Attention Mechanism (Part 2)

Transformers

Definition

Transformers are Deep Learning architectures that completely remove Recurrent Neural Networks (RNNs) and process sequences using the **Attention Mechanism**.

Unlike RNNs, Transformers process all words **in parallel**, making training much faster while capturing long-range dependencies more effectively.

Why Transformers?

Transformers solved the remaining limitations of RNNs and Attention-based Encoder–Decoder models.

They provide:

- Direct word-to-word attention
 - Full parallel training
 - Better long-range dependency learning
 - Cleaner stacked architecture
 - Higher training efficiency
-

Applications of Transformers

Transformers are widely used in modern Artificial Intelligence.

Examples:

- Text → Text
- Image → Text (Image Captioning / Image Summary)
- Text → Image
- Machine Translation
- Question Answering
- Summarization

- Chatbots

Today, almost every Large Language Model (LLM) is based on the Transformer architecture.

Transformer Architecture

Transformers are based on an **Encoder–Decoder Architecture**.

They consist of:

Multiple Encoder Layers



Multiple Decoder Layers

Both Encoder and Decoder usually contain the **same number of stacked layers**.

Layer-wise Understanding

As the model becomes deeper:

Lower Layers

Capture basic word relationships.

Middle Layers

Understand contextual information between words.

Higher Layers

Capture semantic meaning and abstract concepts.

Encoder Layer

Definition

An **Encoder Layer** is the main building block of the Transformer Encoder.

Each encoder layer contains two major components:

1. Multi-Head Self-Attention Layer
2. Feed Forward Neural Network (FFN)

Each encoder layer gradually improves the contextual understanding of every word.

Decoder Layer

Definition

A **Decoder Layer** generates the output sequence using information from both the previous generated words and the encoder outputs.

Each Decoder Layer contains three components:

1. Masked Multi-Head Self-Attention
2. Cross Attention (Encoder–Decoder Attention)
3. Feed Forward Neural Network

Multi-Head Self-Attention Layer

Definition

The **Multi-Head Self-Attention Layer** is the heart of the Transformer architecture.

It allows every word in a sentence to look at every other word simultaneously from multiple

perspectives in order to understand context and relationships before making a prediction.

Example

Sentence:

Please study man

The sentence is first converted into embeddings and then passed to the Self-Attention Layer.

This layer produces **Contextual Embeddings**, meaning every word now understands its relationship with the other words in the sentence.

Embedding

Definition

An **Embedding** is the vector representation of a word.

Instead of storing words as text, Transformers convert every word into numerical vectors before processing them.

Query (Q)

Definition

A **Query** represents what a word is trying to find from the rest of the sentence.

It acts like a question:

"Which other words are important for my meaning?"

Each token generates a Query Vector.

The Query is compared with the Keys of every other word.

A stronger match means higher attention.

Key (K)

Definition

A **Key** describes the information contained inside a word.

Every token generates its own Key Vector.

The Query compares itself with every Key to determine which words are important.

In simple words:

The Key answers the question:

"Do I have the information you're looking for?"

Value (V)

Definition

A **Value** stores the actual information that should be passed to another word once attention has been calculated.

Queries decide **where to look**.

Keys decide **what is relevant**.

Values provide **the actual information**.

Simple Understanding

Sentence



Please Study Man

Embedding



Vector Representation



Self-Attention Layer



Query

Key

Value

Step 1 — Creating Query, Key and Value (QKV)

Suppose the word vectors are:

[1,0] [0,2] [1,1]

The Self-Attention Layer creates three different vectors:

- Query
- Key
- Value

Query Matrix

W_q

[1 0

0 1]

Query Calculation

Please Vector

×

W_q

Key Matrix

W_k

$\begin{bmatrix} 1 & 1 \end{bmatrix}$

$\begin{bmatrix} 0 & 1 \end{bmatrix}$

Calculate:

Please $\times W_k$

Study $\times W_k$

Man $\times W_k$

Value Matrix

W_v

$\begin{bmatrix} 1 & 0 \end{bmatrix}$

1 1]

Calculate:

Word Vector $\times W_v$

Step 2 — Raw Attention Score

Definition

The **Raw Attention Score** measures how important every other word is for the current word.

Formula:

$$\text{Score} = Q \times K^T$$

(Query $\times$ Transpose of Key)

Example

Suppose

Please Query

$$Q = [1, 0]$$

Keys

Please

[1,1]

Study

[0,2]

Man

[1,2]

Now compute the Dot Product between the Query and every Key.

Similarly,

Study and Man also calculate their own attention scores.

Finally,

all these values create a **Relationship Map** of the sentence.

Step 3 — Applying Softmax

Raw Attention Scores are simply numerical values.

They must be converted into probabilities.

For this purpose we apply the **Softmax Function**.

Example

Study

Raw Scores

[2,4,4]

After Softmax

[0.06,0.47,0.47]

Interpretation

Study gives:

6% attention to Please

47% attention to Study

47% attention to Man

The same calculation is performed for every word.

Step 4 — Creating Context Vector

Definition

The **Context Vector** is the final representation of a word after combining information from all related words according to their attention weights.

Formula

Context Vector

=

Attention Weights × Values

Example

Study

Attention Weights

[0.06,0.47,0.47]

Values

[1,0]

$[2,2]$

$[2,1]$

Calculation

$[0.06,0]$

+

$[0.94,0.94]$

+

$[0.94,0.47]$

Final Contextual Vector

$[1.94,1.41]$

Original Word

Study



Self-Attention Layer



Contextual Vector

[1.94,1.41]

Now the word is influenced by every other important word in the sentence.

Complete Self-Attention Summary

Step 1

Create Query, Key and Value



Step 2

Calculate Raw Attention Scores



Step 3

Apply Softmax



Step 4

Generate Context Vector

Why Multi-Head Self-Attention?

A sentence contains much more than words.

It contains:

- Grammar
- Context
- Meaning
- Relationships
- Structure

A single attention head cannot capture all these aspects.

Therefore,

Transformers create **multiple attention heads**, where each head learns different types of relationships.

This is called **Multi-Head Self-Attention**.

Before Feed Forward Neural Network

After Multi-Head Self-Attention, three additional operations are performed.

1. Concatenation

The outputs of all attention heads are combined together.

Concat(Heads)

2. Linear Projection

The concatenated output is multiplied by an output weight matrix.

Formula

Concat(Heads)

$\times$

$W(\text{Output})$

3. Layer Normalization

Formula

Z

$=$

$\text{LayerNorm}(\text{Output} + \text{Original Input})$

Layer Normalization

Definition

Layer Normalization scales all values to a common range, making training more stable and faster.

Residual Connection

Definition

Residual Connections preserve the original information by adding the input back to the output.

This prevents important information from being lost in deep networks.

Positional Encoding

Definition

Transformers process all words simultaneously.

Unlike RNNs, they do not naturally know the order of words.

Therefore, **Positional Encoding** is added to tell the model where each word appears in the sentence.

Example

MAS love DS

DS love MAS

Same words,

Different meanings,

Different positions.

Positional Encoding helps the Transformer understand this difference.

Feed Forward Neural Network (FFN)

Definition

The **Feed Forward Neural Network (FFN)** is the second major component of every Encoder Layer.

After Self-Attention builds contextual understanding, the FFN processes each word

individually to extract deeper and more complex features.

Structure

Linear



Activation Function



Linear

Working

The FFN first expands the embedding dimension.

Example

512



2048

This gives the model more capacity to learn complex patterns.

After learning,

it compresses the representation back to its original size.

Example

2048



512

Unlike Self-Attention,

there is **no communication between words** inside the FFN.

Every word is processed independently using the same neural network.

Why Multiple Encoder Layers?

A single encoder layer cannot fully understand language.

Therefore, multiple encoder layers are stacked.

First Layers

Learn nearby relationships.

Middle Layers

Capture long-range dependencies.

Higher Layers

Understand semantic meaning and abstract concepts.

Each layer refines the previous representation, gradually producing richer contextual embeddings.

This is similar to Deep Neural Networks, where increasing depth allows the model to learn more powerful representations.

Part 2 Summary

Input Sentence



Embeddings



Multi-Head Self-Attention



Query, Key, Value



Raw Attention Scores



Softmax



Context Vector



Concatenation



Linear Projection



Residual Connection



Layer Normalization



Feed Forward Neural Network



Contextual Embeddings



Next Encoder Layer